
CIENCIA DE DATOS / Unidad 2 / 2.4

Selección y extracción de características

No todas las columnas ayudan. Algunas son irrelevantes, otras redundantes y demasiadas a la vez perjudican al modelo. Aquí se aprende a **quedarse con las características que importan** —seleccionándolas— o a **condensar la información en menos dimensiones** —extrayéndolas con PCA—.

2.4.1 Selección de características (filtro, envoltura, embebido)

2.4.2 Extracción de características: PCA

2.4.3 Reducción de dimensionalidad para visualización

2.4 Selección y extracción de características

Tras limpiar, codificar y escalar (sección 2.3), solemos quedar con muchas columnas —a menudo decenas o cientos—. Una **característica** (o *feature*) es cada una de esas variables de entrada que el modelo usa para predecir. Es tentador pensar que "cuantas más, mejor", pero no es así: características irrelevantes añaden ruido, características redundantes confunden, y un exceso de ellas provoca un problema con nombre propio.

DEFINICIÓN

Maldición de la dimensionalidad. A medida que crece el número de características (dimensiones), el espacio se vuelve tan vasto que los datos quedan dispersos: las distancias entre puntos pierden significado, se necesitan muchísimos más datos para cubrir el espacio, y los modelos tienden a *sobreajustar* (memorizar el ruido en vez de aprender el patrón).

Reducir el número de características bien elegido aporta varios beneficios simultáneos:

- **Menos sobreajuste:** menos dimensiones donde el modelo pueda "memorizar" ruido.
- **Entrenamiento más rápido y modelos más ligeros.** Menos columnas, menos cómputo y memoria.
- **Mayor interpretabilidad:** es más fácil entender un modelo con pocas variables claras.
- **A veces, mayor exactitud:** quitar ruido puede mejorar el rendimiento, no solo la velocidad.

Existen dos caminos para lograrlo, y conviene no confundirlos porque la diferencia es conceptual:

	Selección	Extracción
Qué hace	Elige un subconjunto de las características originales y descarta el resto.	Combina las originales en características nuevas (menos en número).
Resultado	Las mismas variables, pero menos.	Variables distintas, derivadas.
Interpretabilidad	Alta: sabes qué columnas quedaron.	Menor: las nuevas son mezclas.
Ejemplo típico	Quedarse con 5 de 50 columnas.	PCA: 50 columnas → 5 componentes.

NOTA

Cómo elegir. Si necesitas explicar *qué* variables influyen (por ejemplo, ante un cliente o un tribunal), prefiere **selección**. Si solo te importa comprimir la información para que el modelo funcione mejor o para visualizar, la **extracción** suele ser más potente.

2.4.1 Selección de características

Seleccionar es escoger, del conjunto original, el subconjunto de características más útil para la tarea. Los métodos se agrupan en tres familias según *cómo* deciden qué conservar.

Métodos de filtro (filter)

Evalúan cada característica con una medida estadística **independiente del modelo** —correlación con el objetivo, varianza, prueba ANOVA, información mutua— y conservan las mejores. Son muy rápidos y sirven como primer cribado.

Lo más básico: descartar columnas casi constantes (varianza nula no aporta nada):

```
In [1]:
from sklearn.feature_selection import VarianceThreshold

vt = VarianceThreshold(threshold=0.0) # elimina columnas constantes
X_filtrado = vt.fit_transform(X)
```

Y quedarse con las k mejores según su relación con la variable objetivo (prueba F de ANOVA para clasificación):

```
In [2]:
from sklearn.feature_selection import SelectKBest, f_classif

sel = SelectKBest(score_func=f_classif, k=2)
X_nuevo = sel.fit_transform(X, y)
sel.get_support() # [False True False True] -> columnas elegidas
```

NOTA

Para variables categóricas se usa `chi2`; para relaciones no lineales, `mutual_info_classif` (información mutua), que captura dependencias que la correlación lineal no ve.

Métodos de envoltura (wrapper)

Usan un modelo real para evaluar **subconjuntos** de características: prueban combinaciones, miden el rendimiento y se quedan con la mejor. Son más precisos que los de filtro pero mucho más costosos. El más común es la **eliminación recursiva de características** (RFE): entrena, descarta la menos importante y repite.

```
In [3]:
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

rfe = RFE(estimator=LogisticRegression(max_iter=1000),
          n_features_to_select=2)
rfe.fit(X, y)
rfe.support_ # máscara de las seleccionadas
rfe.ranking_ # 1 = elegida; >1 = orden de descarte
```

Métodos embebidos (embedded)

La selección ocurre *durante* el entrenamiento del modelo, como parte de su propio funcionamiento. Combinan velocidad y precisión. Dos casos clásicos: la regularización **Lasso** (penalización L1), que lleva a cero los coeficientes de las variables inútiles, y la **importancia de características** de los modelos de árboles.

```
In [4]:
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel

rf = RandomForestClassifier(random_state=42).fit(X, y)
rf.feature_importances_          # importancia de cada característica

sel = SelectFromModel(rf, threshold='median', prefit=True)
X_sel = sel.transform(X)         # conserva las de importancia >= mediana
```

En resumen, las tres familias ofrecen un balance distinto entre velocidad, precisión e interpretabilidad:

Familia	Cómo decide	Balance
Filtro	Estadística por característica, sin modelo.	Muy rápido; ignora interacciones entre variables.
Envoltura	Prueba subconjuntos con un modelo real.	Preciso; costoso y con riesgo de sobreajuste.
Embebido	La selección forma parte del entrenamiento.	Buen término medio; atado al modelo usado.

2.4.2 Extracción de características: Análisis de Componentes Principales (PCA)

Cuando en lugar de descartar columnas queremos **condensarlas**, recurrimos a la extracción. La técnica más usada es el **Análisis de Componentes Principales** (PCA), que crea variables nuevas —los *componentes principales*— como combinaciones de las originales, ordenadas de modo que las primeras capturen la mayor parte de la variación de los datos.

DEFINICIÓN

PCA. Método que rota los ejes del espacio de datos para alinearlos con las direcciones de máxima varianza. El primer componente es la dirección donde los datos más se dispersan; el segundo, la siguiente dirección (perpendicular al primero); y así sucesivamente. Al conservar solo los primeros, reducimos dimensiones perdiendo la menor información posible.

ATENCIÓN

Escalar es obligatorio. PCA se basa en la varianza, y la varianza depende de las unidades. Si una columna está en miles (salario) y otra en decenas (edad), PCA creará que la primera es "más importante" solo por su escala. Aplica siempre `StandardScaler` antes de PCA (enlaza con la sección 2.3.3).

PCA en la práctica: Iris de 4D a 2D

El conjunto Iris tiene 4 características (largo y ancho de sépalo y pétalo). Vamos a proyectarlo a 2 dimensiones para poder visualizarlo, midiendo cuánta información conservamos:

In [5]:

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X, y = load_iris(return_X_y=True)
X_esc = StandardScaler().fit_transform(X) # escalar primero

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_esc)          # ahora X_pca tiene 2 columnas
pca.explained_variance_ratio_
```

Out[5]:

```
array([0.7296, 0.2285]) # 73% + 23% = 96% en solo 2 componentes
```

DEFINICIÓN

Varianza explicada (`explained_variance_ratio_`). Proporción de la variación total de los datos que captura cada componente. Es la brújula para decidir cuántos conservar: aquí, 2 de 4 componentes retienen el 96% de la información.

Al graficar esos dos componentes, las tres especies se separan con notable claridad —a pesar de haber pasado de 4 dimensiones a solo 2—:

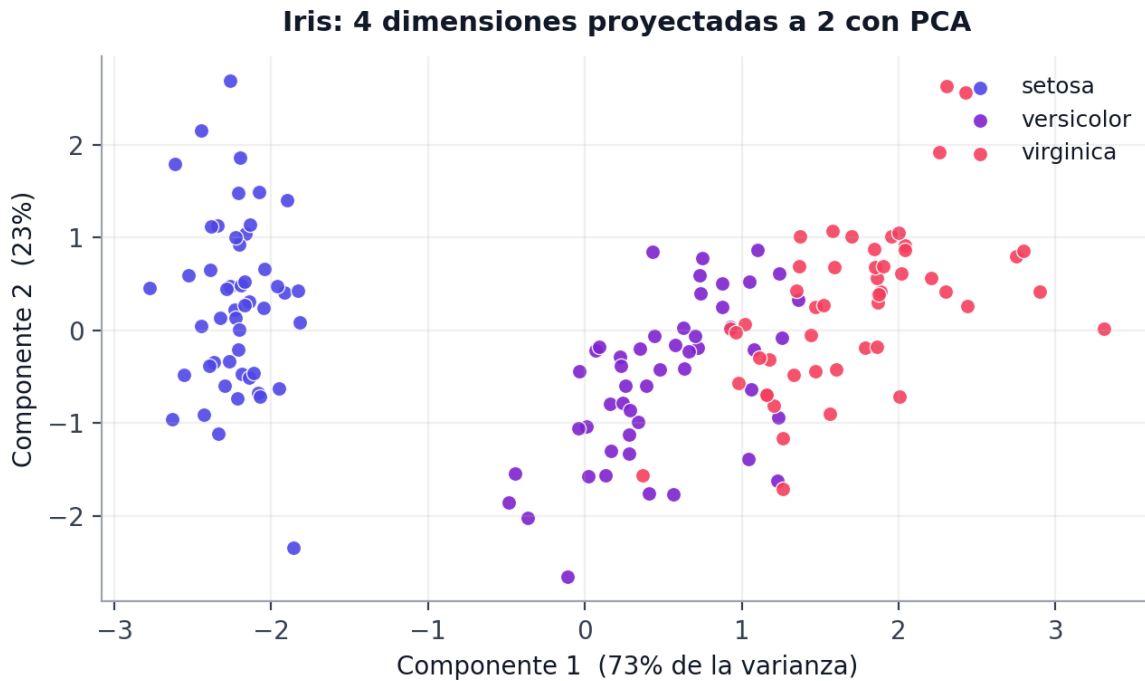


Figura 1. Cada punto es una flor; el color, su especie. Con dos componentes basta para distinguir setosa por completo y casi separar versicolor de virginica.

¿Cuántos componentes conservar?

La estrategia habitual es mirar la **varianza acumulada** y quedarse con los componentes necesarios para alcanzar un umbral (p. ej., 95%). El gráfico de sedimentación (*scree plot*) lo hace evidente:

In [6]:

```
import numpy as np
pca_full = PCA().fit(X_esc)
np.cumsum(pca_full.explained_variance_ratio_)
```

Out[6]:

```
array([0.73, 0.96, 0.99, 1.00]) # con 2 comp. ya superamos el 95%
```

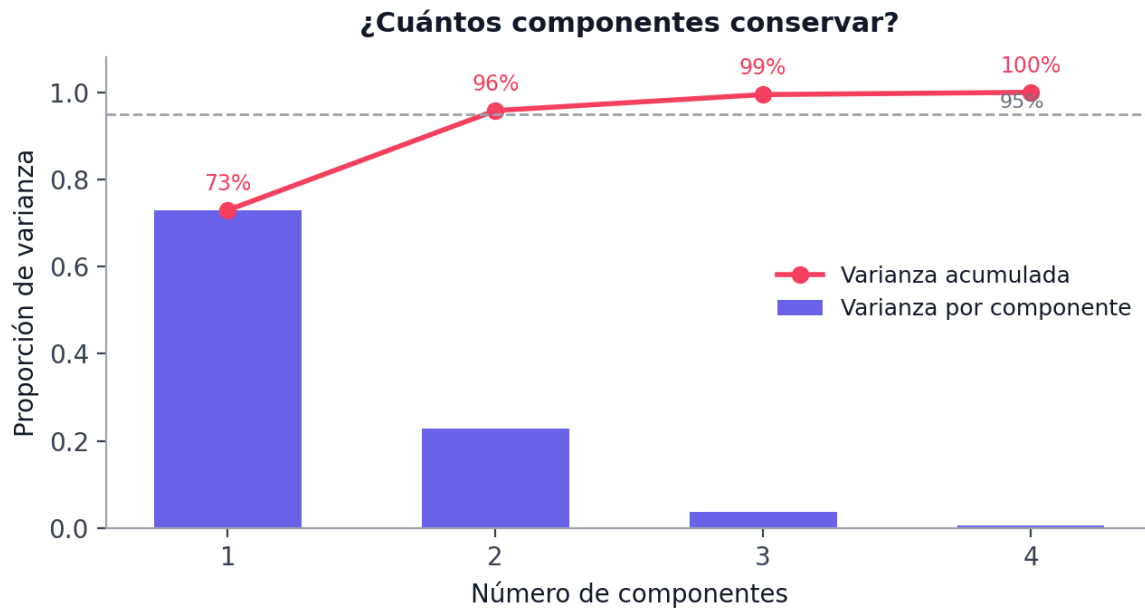


Figura 2. Las barras muestran el aporte de cada componente; la línea, el acumulado. El "codo" tras el 2.º componente indica que los demás aportan poco.

NOTA

El precio de PCA. Los componentes son combinaciones matemáticas de las variables originales, así que dejan de tener un significado directo: "Componente 1" no es una magnitud del mundo real como "ancho de pétalo". Ganamos compresión a cambio de interpretabilidad —el compromiso central de la extracción—.

2.4.3 Reducción de dimensionalidad para visualización

PCA es una técnica **lineal**: proyecta rotando ejes rectos. Funciona muy bien para comprimir, pero cuando la estructura de los datos es compleja y curva, existen métodos **no lineales** pensados específicamente para *ver* datos de alta dimensión en 2D, revelando agrupamientos que PCA podría no mostrar.

t-SNE

t-SNE (*t-distributed Stochastic Neighbor Embedding*) coloca los puntos en 2D procurando que los que eran vecinos en el espacio original sigan cerca. Es excelente para descubrir grupos, pero tiene reglas de uso importantes.

In [7]:

```
from sklearn.manifold import TSNE

X_tsne = TSNE(n_components=2, perplexity=30,
              random_state=42).fit_transform(X_esc)
```

ATENCIÓN

Cómo (no) leer un gráfico t-SNE.

- **Solo sirve para visualizar:** no se usa como entrada de un modelo.
- **El tamaño de los grupos y las distancias entre ellos** no son literales; solo importa qué puntos se agrupan con cuáles.
- **Es estocástico:** cada corrida difiere. Fija `random_state` y prueba varios valores de `perplexity`.

UMAP

UMAP (*Uniform Manifold Approximation and Projection*) persigue un objetivo similar, pero suele ser **más rápido** y preservar mejor la **estructura global** (las distancias entre grupos son algo más fiables que en t-SNE). No viene en scikit-learn; se instala aparte (`pip install umap-learn`).

In [8]:

```
import umap

reductor = umap.UMAP(n_components=2, random_state=42)
X_umap = reductor.fit_transform(X_esc)
```

	PCA	t-SNE / UMAP
Tipo	Lineal	No lineal
Uso principal	Comprimir para modelar o visualizar.	Visualizar agrupamientos.
¿Alimentar un modelo?	Sí, es habitual.	No; solo exploración visual.
Determinista	Sí.	No (depende de la semilla).

Síntesis

Ante demasiadas características, tienes dos caminos. La **selección** conserva un subconjunto de las variables originales —por filtro (rápido), envoltura (preciso) o embebido (equilibrado)— y mantiene la interpretabilidad. La **extracción** crea variables nuevas más compactas: PCA para comprimir (recordando escalar antes y mirar la varianza explicada), y t-SNE o UMAP para visualizar estructuras complejas. La regla de fondo es la misma que en todo el preprocesamiento: aprende los parámetros solo del entrenamiento y aplica la misma transformación al resto.